ST1630 SISTEMAS INTENSIVOS EN DATOS · EAFIT · 2026-2

FraudLens

Guía de estudio completa: el problema, los datos, la arquitectura, cada tecnología, el flujo de punta a punta, las decisiones y sus porqués, los resultados y cómo defenderlo.

Andrés Felipe Vélez Álvarez · Samuel Samper Cardona · Sebastián Salazar Henao · Hellen Yanes Doria

Repositorio: https://github.com/AndresVelez31/FraudLens

Cómo usar esta guía: léanla completa una vez; después, cada uno profundiza en su bloque (sección 18), pero **todos deben poder explicar todo**: en la defensa el profesor pregunta a propósito sobre lo que cada uno no implementó.

Contenido

1. El problema y lo que pidió el cliente
2. Los datos: IEEE-CIS
3. La arquitectura de un vistazo
4. El viaje de un pago, paso a paso
5. Las tecnologías, una por una (qué es, para qué la usamos y por qué)
6. El streaming por dentro: un micro-lote
7. Las reglas de fraude y el modelo de IA
8. El lakehouse: Bronze, Silver y Gold
9. La orquestación con Airflow
10. Las alertas y los correos
11. El consumo: Athena, las preguntas de negocio y Power BI
12. Confiabilidad: exactamente una vez, CAP y tolerancia a fallos
13. Cómo medimos el acierto y qué resultados obtuvimos
14. La latencia: lo que no cumplimos y por qué
15. AWS: despliegue, operación y costo
16. Qué cambió frente al E1 y lecciones aprendidas
17. La demo en vivo
18. Preguntas de defensa y reparto de estudio
19. Glosario

1. El problema y lo que pidió el cliente

Una billetera digital colombiana (tipo Nequi o Daviplata) procesa millones de pagos. Una parte pequeña es fraude: tarjetas robadas, cuentas tomadas, compras con dispositivos ajenos. El área de riesgo necesita dos cosas a la vez, y por eso el problema no cabe en una base de datos relacional simple:

- **Saber ya:** marcar un pago sospechoso en segundos o minutos, mientras todavía se puede bloquear la cuenta o llamar al cliente.
- **Entender el histórico:** analizar meses de pagos para saber dónde y cuándo ocurre el fraude, y ajustar las reglas, sin cargar la base operacional que atiende a los clientes.

El volumen (miles de pagos por segundo en hora pico), la mezcla de tiempo real con análisis histórico y la necesidad de no perder ni duplicar nada justifican una arquitectura de datos distribuida.

Las preguntas que responde el sistema

Id	Pregunta	Dónde se responde
P1–P3	¿Este pago es sospechoso? (ráfaga de pagos, dispositivo nuevo con monto inusual, comercio o dispositivo en lista de riesgo, puntaje del modelo)	En tiempo real: reglas R1–R4 en Spark
P4	¿Cuál es la tasa de fraude por línea de producto, región y franja horaria, semana a semana?	Gold + Athena + Power BI
P5	¿Cuánto tarda una alerta y cuántas dispara cada regla?	Gold (latencia por tramo) + Athena
P6	¿Cuáles son las 10 regiones con más monto sospechoso en los últimos 7 días?	Gold + Athena
P7 (nueva en E2)	¿Cuánto aciertan las alertas contra el fraude real? (precisión, cobertura, falsos positivos)	Gold (cruce con la etiqueta) + Athena + Power BI

Lo que pidió el profesor (como cliente) para el E2

Pedido	Cómo lo resolvimos
Usar datos reales	590.540 pagos reales del dataset IEEE-CIS (Vesta, Kaggle), con la etiqueta de fraude
Medir el acierto contra la realidad	Pregunta P7 y página de Power BI «Acierto contra la realidad»
Power BI	Modo Import sobre Athena: se consulta una vez y se navega gratis
AWS con el mínimo consumo	Plan gratuito, dos EC2 que se apagan solas, servicios que cobran por uso; la carga completa costó unos USD 1,20
IA (extra)	Modelo LightGBM como regla R4

Requisitos del curso que el pipeline debe cumplir

Componente obligatorio	En FraudLens
Streaming real (no simulado con batch)	Kafka + Spark Structured Streaming con micro-lotes de 5 s
Spark con transformaciones no triviales	Joins stream-static (R2, R3), agregaciones por cliente (R4), ventanas deslizantes con watermark (R1), MERGE, esquema en estrella y SCD2 en Gold
Lakehouse Bronze/Silver/Gold en formato abierto, Gold dimensional	Delta Lake sobre S3; Gold en estrella consultable por Athena
Orquestación	Airflow con dos DAG y dependencias
Un componente libre justificado	DynamoDB (alertas) y Athena (consultas)

2. Los datos: IEEE-CIS

No existe un dataset público de pagos colombianos con el fraude etiquetado. Usamos **IEEE-CIS Fraud Detection**, una competencia de Kaggle con pagos reales de la empresa Vesta.

590.540	3,5 %	217.850	80 / 20
pagos en 182 días	son fraude confirmado	clientes	partición por tiempo

Cómo se adaptó al contrato del evento

- **Cliente:** IEEE-CIS no trae un id de cliente. Lo construimos con `card1 + addr1 + D1n` (tarjeta, dirección de facturación y la fecha de primera transacción normalizada). Es la misma clave que usó el ganador de Kaggle.
- **Comercio:** la línea de producto (`ProductCD`: W, C, R, H, S). **Región:** la dirección de facturación (`addr1`).
- **Hora:** el dataset trae segundos relativos (`TransactionDT`); se suman a una fecha base (1 de mayo de 2026).
- Cada evento lleva `ts_evento` (la hora «histórica» del pago) y `ts_emision` (la hora real en que se envió hoy). Las reglas usan la primera; la latencia se mide con la segunda.

Lo único simulado: cuándo llega cada pago

El **replayer** (`producer/replayer.py`) lee los pagos en su orden real y los publica en Kafka acelerados ×1.300 (unos 50 por segundo). El contenido es real; solo comprimimos el tiempo.

> **La etiqueta de fraude nunca viaja por Kafka.** Se guarda aparte (`etiquetas.csv.gz`) y Gold la cruza solo para medir. El sistema decide sin conocer la respuesta, como pasaría en un banco real (el contracargo llega semanas después).

Partición temporal 80/20

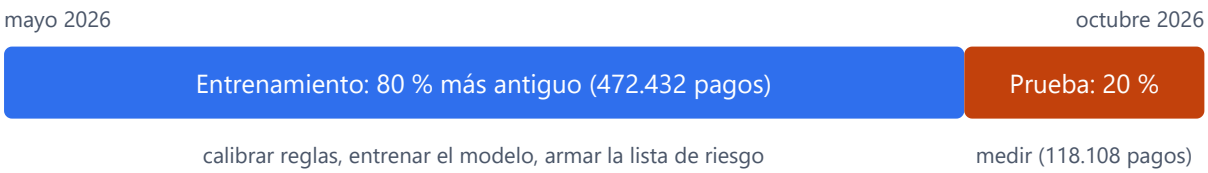


Figura 1. Se aprende con el pasado y se mide con el futuro, nunca al revés.

¿Por qué no una partición aleatoria? Porque mezclaría pagos futuros en el entrenamiento: el modelo «vería el futuro» y el resultado saldría inflado. Con la partición temporal, la prueba se parece a lo que pasaría con pagos que aún no han llegado.

3. La arquitectura de un vistazo

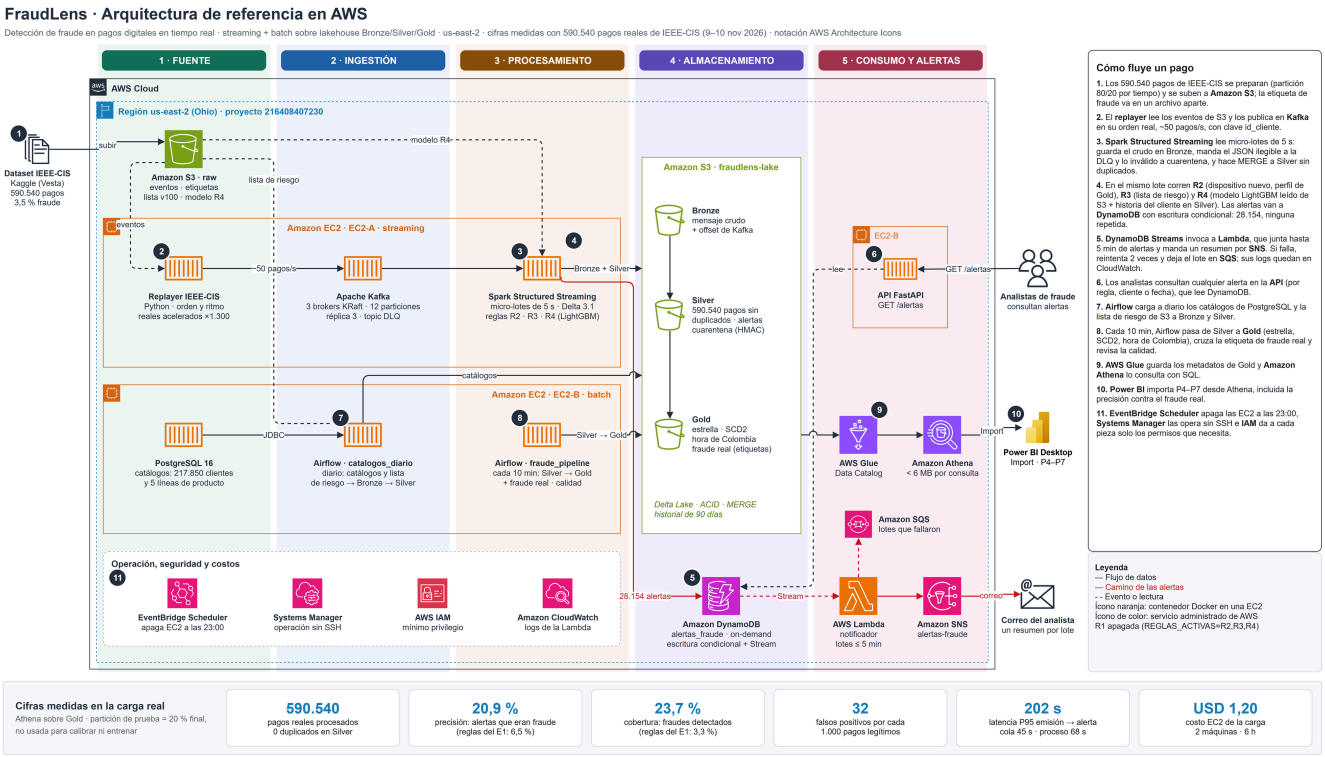


Figura 2. Arquitectura en AWS (us-east-2), en cinco etapas y once pasos numerados. Editable en `docs/entregable2/arquitectura.drawio`.

Se lee de izquierda a derecha, en cinco etapas:

Etapa	Qué hay	Dónde corre
1. Fuente	El replayer de IEEE-CIS (pagos) y PostgreSQL (catálogo de clientes y comercios) + CSV de listas de riesgo	EC2-A (replayer), EC2-B (PostgreSQL)
2. Ingestión	Kafka con 3 brokers, topic `transacciones` (12 particiones) y su DLQ	EC2-A, en contenedores
3. Procesamiento	Spark Structured Streaming (tiempo real) y Spark batch dentro de Airflow (Gold)	EC2-A y EC2-B
4. Almacenamiento	Delta Lake en S3 (Bronze, Silver, Gold) y DynamoDB (alertas)	Servicios administrados
5. Consumo	Lambda + SNS (correos), SQS (fallos), Glue + Athena (SQL), Power BI, API FastAPI	Servicios administrados + EC2-B (API)

> **Regla de costo (desde el E1):** un servicio administrado se usa solo si cobra por uso (S3, DynamoDB, Lambda, SNS, SQS, Glue, Athena). Lo que cobraría por hora aunque nadie lo use (Kafka, Spark, Airflow, PostgreSQL) corre en contenedores Docker sobre dos EC2 que se apagan solas a las 11 p. m.

4. El viaje de un pago, paso a paso

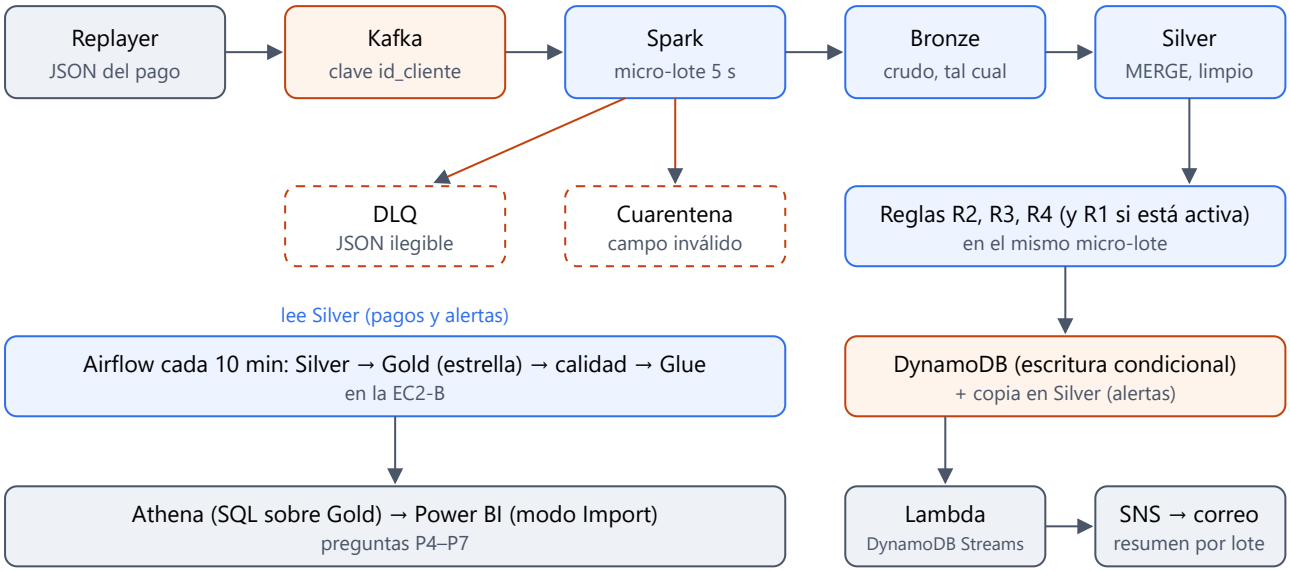


Figura 3. El camino de un pago: arriba el tiempo real (segundos a minutos), abajo el análisis (cada 10 minutos).

1. **Emisión.** El replayer arma el JSON del pago (sin la etiqueta) y lo publica en Kafka con clave `id_cliente`. Todos los pagos de un cliente caen en la misma partición y en orden.
2. **Kafka** lo guarda replicado en 3 brokers; solo confirma al productor cuando al menos 2 tienen la copia.
3. **Spark**, cada 5 segundos, toma lo nuevo de Kafka (un micro-lote) y:
   - escribe todo tal cual en **Bronze**;
   - manda a la **DLQ** (un topic de Kafka) lo que no es JSON;
   - manda a **cuarentena** lo que es JSON pero tiene campos inválidos (monto negativo, tipo inexistente, cliente vacío), con el motivo;
   - hace **MERGE** de lo válido a **Silver** (si ya existía, no lo duplica), con los datos personales seudonimizados;
   - corre las **reglas** sobre los pagos del lote.
4. **Alertas.** Cada alerta se escribe en **DynamoDB** con una escritura condicional (si ya existe, se ignora) y una copia en Silver.
5. **Correo.** DynamoDB Streams avisa a una **Lambda**, que junta las alertas nuevas y manda **un correo resumen por SNS**. Si falla después de 2 reintentos, el lote queda en una cola **SQS**.
6. **Gold.** Cada 10 minutos, **Airflow** construye Gold (dimensiones y hechos), cruza con la etiqueta real, controla la calidad y registra las tablas en **Glue**.
7. **Consumo. Athena** consulta Gold con SQL y **Power BI** importa esos resultados para los tableros. La **API** deja consultar alertas por cliente o por regla.

5. Las tecnologías, una por una

Para cada una: qué es, para qué la usamos y por qué la elegimos frente a las alternativas. Estas son las respuestas a «¿por qué X y no Y?».

Apache Kafka (ingestión)

Qué es: un registro distribuido de mensajes (un «log» que solo se agrega al final), dividido en *topics* y estos en *particiones*. Los productores escriben, los consumidores leen a su ritmo y pueden volver a leer desde cualquier punto (offset).

Cómo lo usamos: 3 brokers en modo KRaft (sin ZooKeeper), topic `transacciones` con 12 particiones y factor de réplica 3, `min.insync.replicas=2`; productor con `acks=all` e idempotencia; clave `id_cliente`; topic `transacciones_dlq` para lo ilegible. Retiene 7 días.

Por qué: desacopla a quien produce de quien procesa (si Spark se cae, los pagos esperan en Kafka sin perderse), permite reprocesar y mantiene el orden por cliente. **Alternativa:** Kinesis o MSK (Kafka administrado), pero no están en el plan gratuito de AWS. Migrar a MSK es cambiar solo la dirección de los brokers.

Apache Spark Structured Streaming (procesamiento)

Qué es: un motor de procesamiento distribuido. Structured Streaming trata un flujo como una tabla que crece y lo procesa en **micro-lotes**.

Cómo lo usamos: Spark 3.5.1; un micro-lote cada 5 s con `foreachBatch` y **checkpoint** en S3 (ahí guarda hasta qué offset de Kafka procesó). El mismo código de `common/` sirve para streaming y para batch.

Por qué: un solo motor para tiempo real y para Gold, escritura nativa a Delta Lake con MERGE y la latencia de micro-lote alcanza para una meta de minutos. **Alternativa:** Flink, con menor latencia por evento y estado propio; habría sido mejor para la latencia (sección 14), pero suma un segundo motor.

Delta Lake sobre Amazon S3 (almacenamiento)

Qué es: un formato de tabla abierto: archivos Parquet en S3 más un registro de transacciones (`_delta_log`) que da operaciones **ACID**, **MERGE**, esquema obligatorio y **viaje en el tiempo**.

Cómo lo usamos: Delta 3.1.0 en el bucket `fraudlens-lake-216408407230`; historial de 90 días; escrituras idempotentes con `txnAppId` / `txnVersion`.

Por qué: S3 es barato y durable, pero por sí solo no tiene transacciones ni actualizaciones; Delta se las da. **Alternativa:** Apache Iceberg (equivalente); elegimos Delta por su integración directa con Spark.

Apache Airflow (orquestación)

Qué es: un orquestador: define flujos como grafos de tareas (DAG) en Python, con horario, dependencias, reintentos y avisos.

Cómo lo usamos: Airflow 2.9 con LocalExecutor en la EC2-B; dos DAG (`catalogos_diario` y `fraude_pipeline`); 3 reintentos con espera exponencial y aviso por SNS si una tarea agota los reintentos.

Por qué: es la referencia del curso, modela dependencias de forma explícita y un fallo detiene lo que sigue. **Alternativa:** Step Functions o MWAA (Airflow administrado, que cobra por hora).

PostgreSQL (fuente operacional)

Simula la base del negocio: catálogo de clientes (217.850, con datos personales) y comercios. Airflow lo extrae a diario por JDBC. Los datos personales **solo existen aquí**: al lake llegan seudonimizados con HMAC-SHA-256.

Amazon DynamoDB (almacén de alertas; el componente libre)

Qué es: una base NoSQL clave-valor administrada, que cobra por uso y responde en milisegundos.

Cómo lo usamos: tabla `alertas_fraude` con clave de partición `id_alerta`; índices secundarios `por_cliente` (`id_cliente` + `ts_alerta`) y `por_regla` (`regla` + `ts_alerta`); **Streams** activado (avisa de cada alerta nueva) y TTL.

Por qué: la alerta se escribe con una **escritura condicional** (`attribute_not_exists(id_alerta)`): si ya existe, no entra. Eso garantiza que una alerta nunca se duplique ni mande dos correos. En términos de CAP es AP en las lecturas (eventualmente consistentes). **Alternativa:** MongoDB (en el E1 inicial), que habría que operar en una EC2.

AWS Lambda, SNS y SQS (notificaciones)

- **Lambda** (`fraudlens-notificador`): función sin servidor que se despierta con DynamoDB Streams, junta las alertas nuevas (hasta 5 min o 1.000 registros) y arma un resumen.
- **SNS** (`alertas-fraude`): publica el resumen y lo entrega por correo a los suscritos.
- **SQS** (`fraudlens-notificador-fallos`): si la Lambda falla tras 2 reintentos, el lote queda aquí para reprocesarlo (cola de mensajes fallidos).

Por qué: cobran por uso y separan el envío de correos del streaming: Spark ya no espera a SNS y no se pierde el aviso si Spark se cae.

AWS Glue y Amazon Athena (consulta)

Glue Data Catalog guarda el esquema de las tablas Gold (una tabla por cada carpeta Delta). **Athena** ejecuta SQL sobre esos archivos en S3 y cobra por dato leído. Las consultas de FraudLens leen menos de 6 MB y tardan unos 3 s. El workgroup `fraudlens` limita cada consulta a 1 GB.

Power BI (tableros)

Se conecta a Athena por ODBC en **modo Import**: trae los datos una vez al actualizar y después se navega sin volver a consultar. Así no se paga cada clic y el tablero funciona con las EC2 apagadas. El modo DirectQuery consultaría Athena en cada interacción.

FastAPI (API de alertas)

Servicio en la EC2-B: `GET /alertas?id_cliente=…&regla=…&desde=…` consulta DynamoDB por los índices; `/salud` verifica la conexión. Documentación automática en `/docs`.

LightGBM (el modelo de la regla R4)

Árboles de decisión con *gradient boosting*: muchos árboles pequeños, cada uno corrige los errores de los anteriores. En datos tabulares sigue superando a las redes neuronales (Grinsztajn et al., 2022) y puntúa rápido. Detalle en la sección 7.

Infraestructura: EC2, Docker, Systems Manager y EventBridge

- **EC2:** dos máquinas m7i-flex.large (2 vCPU, 8 GB). **EC2-A**: 3 brokers de Kafka + Spark Streaming + replayer. **EC2-B**: PostgreSQL + Airflow (con Spark batch) + API.
- **Docker Compose:** un solo `docker-compose.yml` con perfiles `streaming` y `batch`; en local se le suma `compose.local.yml` (MinIO como S3 y DynamoDB Local).
- **Systems Manager (SSM):** todo se opera sin SSH ni llaves; los comandos y los túneles a Airflow y a la API van cifrados por SSM.
- **IAM:** el rol `fraudlens-ec2` da a las EC2 permisos mínimos (S3, DynamoDB, Glue, SNS); no hay llaves en el código.
- **EventBridge Scheduler** (`fraudlens-apagar-ec2`): detiene las EC2 todos los días a las 11 p. m. (Colombia).

6. El streaming por dentro: un micro-lote

El corazón es `streaming/job.py`. Cada 5 segundos, Spark ejecuta `procesar_lote` sobre lo que llegó de Kafka:

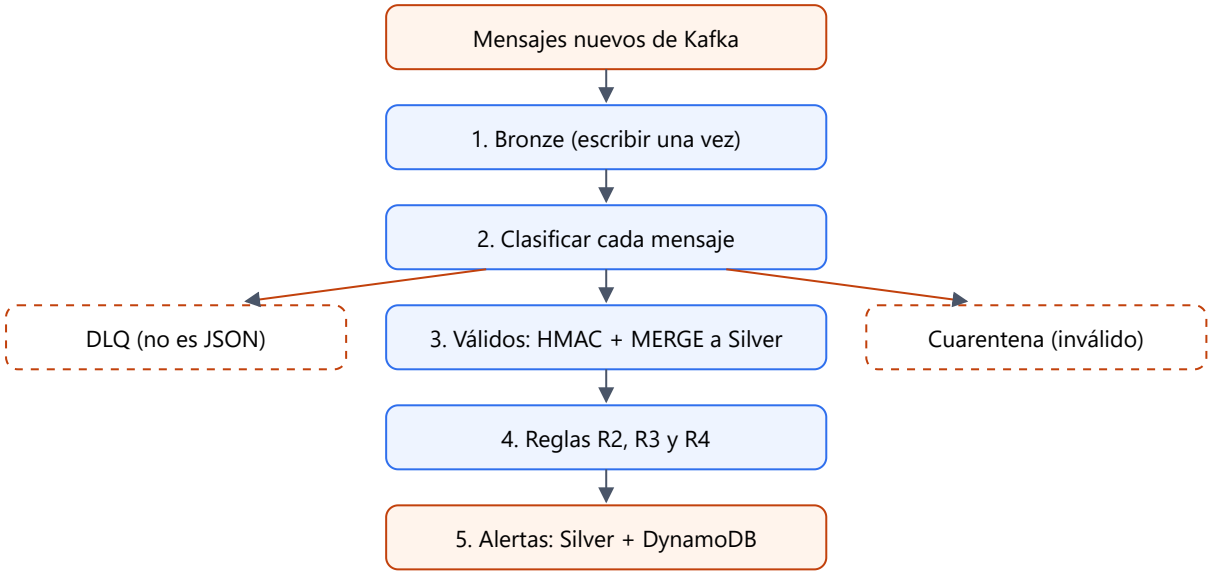


Figura 4. Un micro-lote. Todo lo que escribe es idempotente: si se repite, no duplica.

- **Bronze** guarda el mensaje crudo con su `kafka_topic`, `kafka_partition`, `kafka_offset`, `kafka_timestamp` e `ingest_ts` (hora de entrada al streaming). Es la evidencia: siempre se puede volver a construir todo desde aquí (trazabilidad).
- **Silver** tiene un pago por `id_transaccion`, tipado, validado y con los datos personales seudonimizados (HMAC). El MERGE por `id_transaccion` + `fecha` hace que reenviar un pago no lo duplique.
- **Las reglas** corren sobre los pagos válidos del lote. R2 y R3 hacen un **join stream-static**: unen el micro-lote (stream) con tablas fijas (el perfil del cliente en Gold, los dispositivos ya vistos, la lista de riesgo vigente). R4 lee de Silver la historia de los clientes del lote para calcular sus 17 variables.
- **R1** es una segunda consulta de streaming (`r1_ventanas`) que lee Silver como stream y mantiene ventanas deslizantes de 5 minutos por cliente con **watermark**; hoy está apagada por configuración.

Cada lote deja en el log una línea como esta, que es lo que se muestra en la demo:

`lote 164: raw=2012 validas=2000 cuarentena=9 dlq=3 (242.2s) … · lote 164: 70 alertas R2/R3/R4 (70 nuevas)`

7. Las reglas de fraude y el modelo de IA

Regla	Qué mira (versión `ieee`)	Estado	Precisión en prueba
R1 ráfaga	3 o más pagos del mismo cliente en 5 minutos	Apagada	5,3 %
R2 dispositivo nuevo	Dispositivo nunca usado por el cliente y monto > 1,5× su promedio de 30 días	Activa	13,4 %
R3 lista de riesgo	Comercio o dispositivo en la lista vigente (lista v100: 43 dispositivos aprendidos solo con entrenamiento)	Activa	18,6 %
R4 modelo	LightGBM con 17 variables; alerta si el puntaje supera 0,155	Activa	22,2 %

- Las reglas tienen **versiones** (`RULE_VERSION`: v1 del E1, v2 para reprocesar, `ieee` calibrada) y se encienden o apagan con `REGLAS_ACTIVAS`, sin tocar el código.
- **Calibración:** los umbrales de la versión `ieee` se eligieron con el 80 % de entrenamiento (máximo F1 con a lo sumo 50 alertas por cada 1.000 pagos).
- **R1 se apagó** porque con datos reales casi no acertaba (5,3 %) y producía la mayoría de los falsos positivos.

El modelo R4 en detalle

Las 17 variables (`common/variables.py`) se calculan con **el mismo código** en el entrenamiento y en el streaming, para que lo entrenado y lo que corre sean idénticos:

Grupo	Variables
Del pago	monto, log del monto, tipo (producto), región, dominio del correo, si trae dispositivo, hora, día de la semana
Historia del cliente	pagos en los últimos 5 min, 1 h y 24 h; pagos previos totales; segundos desde el pago anterior; monto promedio de 30 días; monto frente a ese promedio
Dispositivo	si el dispositivo es nuevo para el cliente; pagos previos con ese dispositivo

- Entrenado con el 80 % antiguo (`ml/modelo_lightgbm.py`) y publicado en el lake (`models/r4/`); el streaming lo carga al arrancar.
- **El umbral** (0,155) se eligió para no pasar el presupuesto de alertas (sección 13).
- **AUC-ROC:** 0,806 con las 17 variables del contrato; 0,908 si se le dan las columnas anonimizadas de Vesta; el 1.er lugar de Kaggle llegó a 0,946. La brecha es de **datos**, no de técnica.

8. El lakehouse: Bronze, Silver y Gold

Capa	Qué guarda	Quién escribe
Bronze	Todo lo recibido, tal cual, con su rastro de Kafka. Catálogos crudos de PostgreSQL y CSV	Streaming; DAG `catalogos_diario`
Silver	Pagos limpios y únicos, alertas, cuarentena, catálogos (clientes con HMAC, comercios, lista de riesgo)	Streaming; DAG `catalogos_diario`
Gold	Esquema en estrella listo para analizar	DAG `fraude_pipeline`

El esquema en estrella de Gold

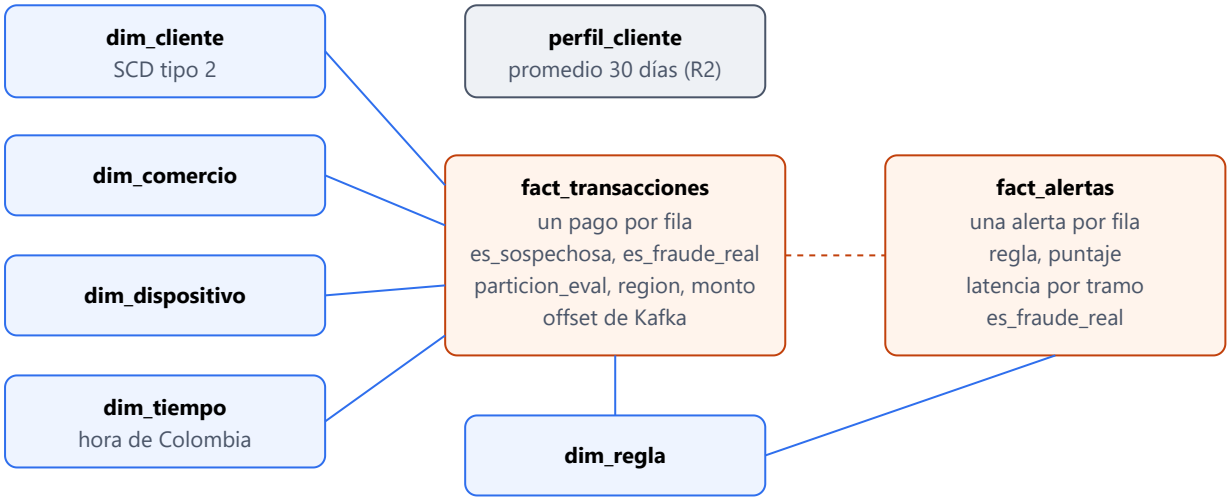


Figura 5. Gold en estrella: los hechos en el centro, las dimensiones alrededor.

- **¿Por qué estrella?** Las preguntas P4–P7 son agregaciones de hechos por dimensiones (por región, por franja horaria, por regla). La estrella hace esas consultas simples y rápidas.
- **SCD tipo 2** en `dim_cliente`: si un cliente cambia de ciudad o segmento, se cierra la versión anterior y se abre una nueva, para analizar cada pago con los datos que el cliente tenía en ese momento.
- **Hora de Colombia:** `dim_tiempo` y las franjas se calculan en `America/Bogota`.
- **La etiqueta real** se cruza en Gold (`es_fraude_real`, `particion_eval`): así P7 mide el acierto sin que el streaming la haya visto.
- **Incremental por llegada:** Gold procesa lo que **llegó** (`ingest_ts`), no por la fecha del pago, porque los pagos históricos llegan hoy. Con una **marca de agua**, cada corrida solo procesa lo que llegó después de la última que ya está en Gold (con una hora de margen).

9. La orquestación con Airflow

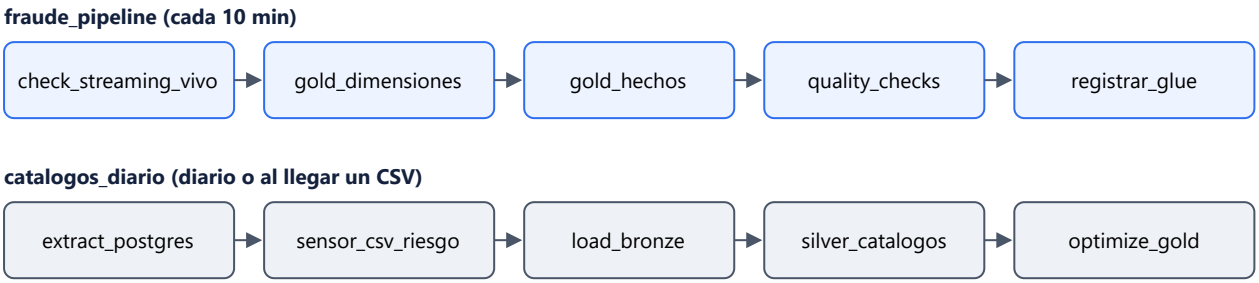


Figura 6. Los dos DAG. Si una tarea falla, las siguientes no corren (`all_success`).

- **check_streaming_vivo** lee el «latido» del streaming y falla si lleva más de 2 minutos sin avanzar: no tiene sentido construir Gold con datos viejos.
- **quality_checks** verifica: tasa de rechazo (DLQ + cuarentena) menor al 1 %, sin duplicados en Silver, en alertas ni en Gold, sin llaves nulas, y que lo que lleva más de 15 min en Silver ya esté en Gold.
- **registrar_glue** registra o actualiza las tablas en Glue; si cambian las columnas, recrea la tabla (antes Athena fallaba con `INVALID_GLUE_SCHEMA`).
- Hay un tercer DAG pequeño (`disparar_catalogos_si_csv_nuevo`) que cada 5 min mira si llegó una lista de riesgo nueva y dispara `catalogos_diario`.
- Límite de 15 min por corrida y una sola corrida a la vez: si una se alarga, la siguiente espera. Una corrida dura unos 7,5 min.

10. Las alertas y los correos



Figura 7. Notificación desacoplada del streaming.

Por qué un resumen y no un correo por alerta

En una prueba temprana, un correo por alerta mandó más de 500 correos en minutos. Un analista no lee eso. Ahora la Lambda agrupa las alertas nuevas y manda un resumen (cantidad, reglas, las de mayor puntaje, hora de Colombia). La Lambda junta como máximo 5 minutos, que es el límite de AWS: una ráfaga produce un correo por cada *shard* del stream.

Qué decirle al profesor sobre cómo funcionan las alertas

- La alerta **se crea en segundos o minutos** y queda consultable al instante en DynamoDB y la API (eso es lo que usaría el analista).
- El correo es solo un **aviso agrupado**; en un banco real el analista trabaja una cola en una aplicación, no en su buzón.
- Una alerta **nunca se duplica**: si Spark reprocesa un lote, la escritura condicional rechaza la repetida y no sale otro correo.
- Si el envío falla, **no se pierde**: el lote queda en SQS (lo probamos forzando un error en la Lambda).
- Un resumen único cada 30 minutos necesitaría un acumulador con EventBridge; se dejó como mejora.

11. El consumo: Athena, las preguntas de negocio y Power BI

Las consultas están en `dashboards/athena/` y funcionan igual en Athena y en Spark SQL (`batch/consultas.py` para local):

Consulta	Qué responde
`p4_tasa_fraude.sql`	% de pagos sospechosos y de fraude real por producto, región y franja horaria, semana a semana
`p5_latencia_alertas.sql`	Latencia promedio, P95 y máxima por regla, partida en envío, cola y proceso
`p6_top10_comercios.sql`	Las 10 regiones (con su producto) con más monto sospechoso en los 7 días anteriores al último pago
`p7_acierto.sql` y `p7_acierto_por_regla.sql`	Precisión, cobertura y falsos positivos por partición y por regla

Power BI (`dashboards/powerbi/FraudLens.pbip`) tiene medidas como Precisión, Cobertura, FP por 1.000 legítimas y Precisión de la regla, y una página «Acierto contra la realidad» filtrada a la partición de prueba.

12. Confiabilidad: exactamente una vez, CAP y tolerancia a fallos

Kafka entrega **al menos una vez**: si algo falla, un mensaje puede llegar dos veces. El efecto **exactamente una vez** lo ponemos nosotros en cada escritura:

Dónde	Mecanismo	Código
Productor → Kafka	`acks=all` + productor idempotente + `min.insync.replicas=2`	`producer/producer.py`
Bronze y cuarentena	Escritura Delta con `txnAppId` / `txnVersion`: si el lote ya se escribió, Delta lo ignora	`common/tables.py` (`escribir_una_vez`)
Silver	MERGE por `id_transaccion` + `fecha`, con reintentos si dos escrituras chocan	`streaming/job.py`, `common/alerts.py`
DynamoDB	`PutItem` con `attribute_not_exists(id_alerta)`; `id_alerta` es un hash determinista de regla + transacción + versión	`common/alerts.py`
Gold	MERGE por llaves; correr dos veces deja el mismo Gold	`batch/gold.py`

> **Prueba real:** durante la carga de 3 horas el replayer se cayó y, al reanudarlo, reenvió 206 pagos. Silver terminó con **590.540 filas exactas** y 0 alertas repetidas.

Si Spark se cae a mitad de un lote

Al reiniciar, lee el checkpoint en S3 y vuelve a procesar el lote que no terminó. Como todas las escrituras son idempotentes, lo que alcanzó a escribir no se duplica. Se probó con `kill -9`.

Si cae un broker de Kafka

Con 3 brokers y `min.insync.replicas=2`, se sigue escribiendo con uno caído. Con dos caídos, el productor **se detiene** y reintenta en lugar de perder datos.

Posición CAP de cada componente

Componente	CAP	Qué sacrifica
Kafka	CP	Si caen 2 brokers, deja de aceptar escrituras para no perder pagos
S3 / Delta	CP	Consistencia fuerte de lectura después de escritura
DynamoDB (lecturas de la API)	AP	Una alerta puede tardar un instante en verse; la API lo declara
Airflow / PostgreSQL	CP	Un solo nodo: si cae, el DAG espera

Mensajes rotos y privacidad

- JSON ilegible → DLQ, con el topic, partición y offset de origen en los encabezados. JSON con campos inválidos → cuarentena con el motivo. Ninguno detiene el pipeline.

- Datos personales (nombre, documento, teléfono, correo) solo en PostgreSQL; al lake entran con **HMAC-SHA-256** con una clave secreta. Un hash simple se revierte probando todos los documentos posibles; con HMAC hace falta la clave.

13. Cómo medimos el acierto y qué resultados obtuvimos

Por qué no la exactitud (accuracy)

Con 3,5 % de fraude, un sistema que dijera «nada es fraude» acertaría el 96,5 % de las veces y no serviría. Medimos:

- **Precisión:** de cada 100 alertas, cuántas eran fraude. Es el trabajo útil del analista.
- **Cobertura (recall):** de cada 100 fraudes reales, cuántos alertamos. Es la plata que se salva.
- **Falsos positivos por cada 1.000 pagos legítimos:** clientes honestos molestados.
- **Presupuesto fijo:** unas 46 alertas por cada 1.000 pagos, lo que un equipo alcanza a revisar. Comparar enfoques con el mismo presupuesto es justo: ganar cobertura alertando todo no vale.

Resultados en la partición de prueba (carga real en AWS)

20,9 %	23,7 %	32	962
precisión (E1: 6,5 %)	cobertura (E1: 3,3 %)	falsos positivos por 1.000 legítimos	fraudes detectados

Enfoque	Precisión	Cobertura
Reglas del E1 tal cual (v1)	6,5 %	3,3 %
Reglas calibradas (`ieee`)	7,4 %	10,0 %
LightGBM solo, fuera de línea	20,9 %	28,2 %
Sistema en streaming real (R2 + R3 + R4)	**20,9 %**	**23,7 %**
Referencia: LightGBM con todas las columnas de Vesta	43,0 %	58,0 %

- **Por producto:** el C tiene 13,4 % de fraude real y detectamos el 40 %; el W tiene 1,9 % y detectamos el 6,4 %.
- **En entrenamiento** la precisión sale 39 %: es el modelo viendo datos que ya conoce. Por eso **solo se reporta la partición de prueba**.

Estado del arte

El 1.er lugar de Kaggle (Chris Deotte y colegas, 2019) llegó a AUC 0,946 con todas las columnas y la misma idea de cliente (`card1 + addr1 + D1`). Nuestro modelo llega a 0,908 con las columnas de Vesta y a 0,806 con las 17 variables del contrato, que es lo que corre en tiempo real. La diferencia es de datos: un banco real sí tendría esas columnas. Detalle en `docs/entregable2/estado_del_arte.md`.

14. La latencia: lo que no cumplimos y por qué

La meta del E1 (RNF-04) era: del pago a la alerta, **menos de 30 s en el P95**. Medimos **~200 s**. Es el único requisito importante que no se cumple, y hay que decirlo de frente.



Figura 8. Latencia partida en tramos (medida en Gold con las marcas de tiempo de Bronze).

- **Cómo se mide:** desde que el pago se emite (`ts_emision`) hasta la alerta, partida en envío (Kafka − emisión), cola (lote − Kafka) y proceso (alerta − lote). Se mide desde la llegada que provocó la alerta, no desde un envío anterior del mismo pago.
- **La causa:** R4 lee de Silver toda la historia de los clientes del lote. Silver tiene 590 mil filas, así que cada lote tarda más de lo que tarda en llegar el siguiente y se forma una cola.
- **El arreglo:** una **tabla de estado por cliente** (conteos y promedios) en memoria o en DynamoDB, actualizada en cada lote, para no leer S3 cada vez. Es lo que hacen las arquitecturas de referencia con el estado de Flink.
- **Lo que sí cumple:** Athena responde en ~3 s y el pago llega a Gold en menos de ~20 min.

15. AWS: despliegue, operación y costo

- **Proyecto** en el plan gratuito de AWS, región **us-east-2**. Kinesis y MSK no están en ese plan; por eso Kafka va en EC2.
- **Recursos:** bucket S3 del lake, base y workgroup `fraudlens` de Glue/Athena, tabla DynamoDB `alertas_fraude`, topic SNS `alertas-fraude`, Lambda `fraudlens-notificador` y su cola SQS, rol `fraudlens-ec2`, grupo de seguridad `fraudlens-sg` y el horario de apagado.
- **Despliegue:** cada EC2 clona el repo en `/opt/fraudlens`, se completa el `.env` (sin llaves de AWS: las credenciales vienen del rol) y se corre `docker compose up -d --build`. Para actualizar: `git pull` y otra vez `up`.
- **Operación sin SSH:** por Systems Manager; los túneles a Airflow (8081) y a la API (8080) van cifrados y no hay puertos de administración abiertos a internet.
- **Costo:** la carga completa (590.540 pagos, ~3 h) costó unos **USD 1,20** de EC2; Athena, S3, DynamoDB y Lambda, centavos. Con las EC2 encendidas son unos USD 0,20 por hora.

16. Qué cambió frente al E1 y lecciones aprendidas

Área	E1	E2	Por qué
Datos	Productor sintético	590.540 pagos reales	Con datos inventados no se mide el acierto
Reglas	R1–R3 a mano	Calibradas, R4 con LightGBM, R1 apagada	Las del E1 acertaban 6,5 %
Correos	Spark → SNS, uno por alerta	Lambda: resumen por lote; fallos a SQS	Una prueba mandó más de 500 correos
Consumo	Metabase en una EC2	Power BI Import sobre Athena	Lo pidió el cliente; no cobra por clic
Plataforma	Learner Lab, 3 EC2, us-east-1	Plan gratuito, 2 EC2 con apagado, us-east-2	El Learner Lab apaga todo cada 4 h
Gold	Por fecha del pago, UTC	Por fecha de llegada, hora de Colombia, con la etiqueta real	Los pagos históricos llegan hoy

Se mantuvo: Kafka con 3 brokers, Spark Structured Streaming, Delta Lake Bronze/Silver/Gold, Airflow y DynamoDB con escritura condicional.

Lecciones de operar el sistema de verdad

- Un correo por alerta inunda el buzón → resumir por lote.
- Un servicio que arranca solo puede releer Kafka desde el principio → `STARTING_OFFSETS=latest`.
- Una carga de 3 h se cae → el replayer se reanuda con `--saltar`.
- Un esquema que cambia rompe Athena → Glue se actualiza solo.
- Una carga grande en un día hacía que Gold reprocesara todo y el DAG pasara su límite → marca de agua y OPTIMIZE diario.

- Reenviar un pago ya cargado distorsiona las métricas → la demo «reserva» los últimos pagos reales y los vuelve a enviar.

17. La demo en vivo

La guía del curso pide ver en vivo: el productor enviando, Spark procesando, una consulta de Gold que responda una pregunta del E1 y el DAG de Airflow. Todo se maneja desde el portátil con `bash scripts/demo_aws.sh <paso>`; el guion completo está en `docs/entregable2/demo.md`.

Paso	Qué hace	Tiempo medido
`encender` / `estado`	Enciende las EC2 y muestra los contenedores	~3 min
`reservar 2000`	Saca del sistema los últimos 2.000 pagos del dataset (antes de la demo)	~2 min
`enviar 2000`	Los vuelve a mandar: llegan con su historia y su tasa real de alertas	4 s
`streaming`	El micro-lote: 2.000 válidos, 9 a cuarentena, 3 a la DLQ, 70 alertas	~3–4 min
`alertas R4`	Las alertas nuevas desde la API	inmediato
`airflow` / `gold`	El DAG en el navegador (túnel) o en la terminal	~7,5 min por corrida
`consulta p6`, `p5`, `p7`	Preguntas de negocio sobre Gold en Athena	~3 s

> **Antes de cada demo hay que correr `reservar 2000`.** Si los pagos ya están cargados, Silver los descarta por repetidos y no salen alertas nuevas (eso, por cierto, demuestra la idempotencia).

18. Preguntas de defensa y reparto de estudio

Las preguntas con respuesta corta y la línea exacta del código están en `docs/entregable2/guia_defensa.md`. Las que la guía del curso anuncia:

- **¿Por qué Kafka y no Kinesis?** Sección 5.
- **Si el volumen se multiplica por 100, ¿qué cambian primero?** 1) La historia de R4 a una tabla de estado; 2) Spark a un clúster (EMR); 3) Kafka a MSK con más particiones; S3, DynamoDB, Lambda y Athena ya escalan.
- **¿Cómo evitan procesar dos veces si Spark falla a mitad?** Sección 12.
- **¿Posición CAP del componente más crítico?** Kafka, CP. Sección 12.
- **¿Qué pasa si cae un broker? Muéstrenlo en el código.** `docker-compose.yml` (réplica 3, `min.insync.replicas=2`) y `producer/producer.py` (`acks=all`, idempotencia).
- **¿Por qué la clave de DynamoDB es `id_alerta` y no `id_cliente`?** Porque es determinista y única: permite la escritura condicional que evita duplicados y reparte la carga parejo; con `id_cliente` un cliente con muchas alertas sería una partición caliente. Las consultas por cliente usan el índice `por_cliente`.

Integrante	Foco de estudio	Archivos
Andrés Felipe Vélez	Kafka, replayer, infraestructura AWS, Lambda, demo	`producer/`, `docker-compose.yml`, `infra/`, `lambdas/`, `scripts/`
Samuel Samper	Streaming, reglas R1–R4, idempotencia, alertas	`streaming/`, `common/rules.py`, `common/r4.py`, `common/tables.py`, `common/alerts.py`
Sebastián Salazar	Gold, calidad, Athena, Power BI, métricas de acierto	`batch/`, `dashboards/`
Hellen Yanes	Airflow, catálogos, PII/HMAC, API, modelo y evaluación	`dags/`, `common/pii.py`, `api/`, `ml/`, `docs/evaluacion/`

19. Glosario

Término	Significado
ACID	Atomicidad, consistencia, aislamiento y durabilidad: una escritura entra completa o no entra
At-least-once / exactly-once	Al menos una vez (puede repetirse) / exactamente una vez (efecto sin duplicados)
AUC-ROC	Qué tan bien ordena el modelo fraudes por encima de legítimos (0,5 = azar, 1 = perfecto)
Broker	Un servidor de Kafka
CAP	En una partición de red, un sistema elige entre consistencia (C) y disponibilidad (A)
Checkpoint	Dónde guarda Spark hasta qué offset procesó, para retomar tras una caída
DLQ	Cola de mensajes muertos: lo que no se pudo leer
Join stream-static	Unir cada micro-lote con una tabla fija (perfil, lista de riesgo)
Marca de agua de Gold	La llegada más reciente ya procesada; Gold solo procesa lo posterior
MERGE	Insertar si no existe, actualizar si existe, en una sola operación
Micro-lote	El grupo de mensajes que Spark procesa junto cada 5 s
Offset	Posición de un mensaje dentro de una partición de Kafka
P95	El valor por debajo del cual queda el 95 % de los casos
Precisión / cobertura	De las alertas, cuántas eran fraude / de los fraudes, cuántos se alertaron
SCD tipo 2	Dimensión que guarda versiones históricas en lugar de sobrescribir
Seudonimizar (HMAC)	Reemplazar un dato personal por un código que no se revierte sin la clave
Watermark (streaming)	Hasta qué hora de evento se esperan datos atrasados antes de cerrar una ventana

Fuentes en el repositorio: README.md, docs/entregable2/ (cambios_frente_e1.md, estado_del_arte.md, guia_defensa.md, demo.md), docs/evaluacion/README.md, docs/datos/ieee-cis.md y el código citado en cada sección.